实验 7 Hebb 学习与 STDP 突触可塑性

7.1 实验目的

1. **理解 Hebb 学习规则的基本思想**：通过比较不同活动组合下的权重变化，理解“共同活动导致连接增强”的含义。

2. **区分 Hebb 学习规则的不同变体**：比较标准 Hebb、基于衰减的 Hebb、突触前门控 Hebb、突触后门控 Hebb 和协变规则在更新方向上的差异。

3. **理解 Hebb 学习中的有界性、异突触抑制与特异化**：观察软边界、硬边界、突触后门控抑制和 BCM 滑动阈值对突触权重演化的影响。

4. **理解 STDP 的时间因果性**：通过改变突触前、突触后脉冲的相对时间，观察长期增强和长期抑制的产生条件。

5. **理解权重依赖 STDP 的含义**：比较经典 STDP 与软边界权重依赖 STDP 在时间窗口形状上的差异。

6. **继续掌握基础仿真与绘图方法**：使用 `numpy` 和 `matplotlib` 完成权重更新、曲线绘制和实验现象分析。

7.2 实验说明

本次实验围绕课堂中讲解的 **Hebb 学习规则** 和 **STDP 学习规则** 展开。实验设置 2 个大类实验：

1. Hebb 学习规则及其变体；

2. STDP 学习规则及其变体。

本次实验建议保存为两个代码文件，并在文中保留 8 处 TODO：

1. 在 Hebb 实验中补全标准 Hebb 更新公式；

2. 在 Hebb 实验中补全基于衰减的 Hebb 更新公式；

3. 在 Hebb 实验中补全突触前门控和突触后门控 Hebb 更新公式；
4. 在 Hebb 实验中补全协变规则；
5. 在 Hebb 实验中补全硬边界和软边界处理；
6. 在 Hebb 实验中补全 BCM 更新公式；
7. 在 STDP 实验中补全 pair-based STDP 分段公式；
8. 在 STDP 实验中补全权重依赖 STDP 的硬边界和软边界幅度；

在开始实验之前，请先激活实验 1 中创建的虚拟环境，并安装本次实验需要用到的库（已经安装可以跳过）：

```
conda activate brain
pip install numpy matplotlib
```

如果校园网下载速度较慢，也可以使用镜像源，例如：

```
pip install numpy matplotlib -i https://pypi.tuna.tsinghua.edu.cn/simple
```

一、Hebb 学习规则及其变体

Hebb 学习规则描述的是突触前神经元和突触后神经元活动之间的相关性如何影响突触权重。其基本形式为：

$$\frac{dw_{ij}}{dt} = F(w_{ij}; v_i, v_j) \qquad (1)$$

其中，w_{ij} 表示从突触前神经元 j 到突触后神经元 i 的连接权重，v_i 表示突触后神经元活动，v_j 表示突触前神经元活动。我们使用每个时间步的权重增量 Δw 来实现 Hebb 学习，并观察不同 Hebb 变体的差异。标准 Hebb 学习规则可写为：

$$\frac{dw_{ij}}{dt} = c_{2corr}(w_{ij}) v_i v_j \qquad (2)$$

如果不考虑边界约束，我们使用下面的离散更新形式：

$$\Delta w_{ij} = \eta v_i v_j \qquad (3)$$

软边界形式为：

$$c_{2corr}(w_{ij}) = \gamma_2(w_{max} - w_{ij}) \qquad (4)$$

对应的权重增量为：

$$\Delta w_{ij} = \gamma_2(w_{max} - w_{ij})v_i v_j \qquad (5)$$

硬边界形式为：

$$c_{2corr}(w_{ij}) = \eta_0 H(w_{max} - w_{ij}) \qquad (6)$$

对应的权重增量为：

$$\Delta w_{ij} = \eta_0 H(w_{max} - w_{ij})v_i v_j \qquad (7)$$

其中，$H(\cdot)$ 为 Heaviside 阶跃函数。当 $w_{ij} < w_{max}$ 时继续增长，当 $w_{ij} = w_{max}$ 时停止增长。

本大类实验包含三个任务：

1. 比较不同 Hebb 变体在四种活动组合下的更新方向。这里的活动组合顺序为突触后活动 `v_i` / 突触前活动 `v_j`，例如 突触后 `ON` / 突触前 `OFF` 表示 $v_i = 1, v_j = 0$；
2. 比较无约束、硬边界、软边界对权重增长的影响；
3. 比较固定阈值和 BCM 滑动阈值下，两组突触前输入的权重特异化过程。

任务三中的 `Group 1` 和 `Group 2` 表示两组不同的突触前输入模式。实验会让两组输入交替出现，并比较固定阈值与滑动阈值 BCM 的差异。滑动阈值先计算输出活动的运行平均 $\langle v_i \rangle$，再令 $\theta(t) = \alpha_\theta \langle v_i \rangle$。该机制会使神经元对强输入组形成更强响应，同时压低弱输入组的连接。

下面给出 `exp7_1.py` 的框架代码。

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle
```

```python
# 配置中文字体，避免绘图时标题或坐标轴出现乱码
plt.rcParams["font.sans-serif"] = [
    "PingFang SC",
    "Hiragino Sans GB",
    "Microsoft YaHei",
    "SimHei",
    "Noto Sans CJK SC",
    "Arial Unicode MS",
    "DejaVu Sans",
]
plt.rcParams["axes.unicode_minus"] = False


# 将权重变化量转成便于观察的方向标记
def delta_to_sign(delta, eps=1e-10):
    if delta > eps:
        return 1
    if delta < -eps:
        return -1
    return 0


# 标准 Hebb 学习：突触前和突触后共同活跃时增强
def standard_hebb_delta(v_i, v_j, eta):
    # TODO 1：
    # 请根据离散更新公式 Δw_ij = eta * v_i * v_j 补全标准 Hebb 更新量。
    # 输入：
    #   v_i：突触后神经元活动；
    #   v_j：突触前神经元活动；
    #   eta：学习率。
    # 输出：
    #   返回一个标量，表示当前时间步的权重变化量。
    raise NotImplementedError("请补全 standard_hebb_delta")


# 基于衰减的 Hebb 学习：共同活动增强，无刺激时权重衰减
def decay_hebb_delta(w, v_i, v_j, gamma2, gamma0):
```

```python
    # TODO 2:
    # 请根据 PPT 中基于衰减的 Hebb 规则补全权重变化量。
    # 连续形式为:
    #   dw_ij/dt = gamma2 * (1 - w_ij) * v_i * v_j - gamma0 * w_ij
    raise NotImplementedError("请补全 decay_hebb_delta")


# 门控 Hebb 学习：比较突触前门控和突触后门控
def gated_hebb_delta(v_i, v_j, eta, v_theta, gate_type):
    # TODO 3:
    # 请补全两种门控 Hebb 规则。
    # 当 gate_type == "pre" 时，使用突触前门控:
    #   Δw_ij = eta * (v_i - v_theta) * v_j
    # 当 gate_type == "post" 时，使用突触后门控:
    #   Δw_ij = eta * v_i * (v_j - v_theta)
    # 如果 gate_type 不是 "pre" 或 "post"，请抛出 ValueError。
    raise NotImplementedError("请补全 gated_hebb_delta")


# 协变规则：关注活动相对平均活动水平的共同变化
def covariance_hebb_delta(v_i, v_j, v_i_bar, v_j_bar, eta):
    # TODO 4:
    # 请根据协变规则补全权重变化量:
    #   Δw_ij = eta * (v_i - v_i_bar) * (v_j - v_j_bar)
    raise NotImplementedError("请补全 covariance_hebb_delta")


# 标准 Hebb 的有界更新量：按 PPT 中 c2corr(w_ij) 的软/硬边界形式实现
def bounded_hebb_delta(w, v_i, v_j, bound_type, eta0=0.06, gamma2=0.06,
 w_max=1.0):
    # TODO 5:
    # 请按 PPT 中 c2corr(w_ij) 的形式补全三种标准 Hebb 更新量。
    # 1. bound_type == "none": Δw_ij = eta0 * v_i * v_j;
    # 2. bound_type == "hard": Δw_ij = eta0 * H(w_max - w) * v_i * v_j;
```

```python
    # 3. bound_type == "soft": Δw_ij = gamma2 * (w_max - w) * v_i * v_j。
    # 注意：这里返回的是当前时间步的 Δw，不是更新后的权重。
    # 如果 bound_type 不是以上三种，请抛出 ValueError。
    raise NotImplementedError("请补全 bounded_hebb_delta")


# BCM 规则：突触后活动超过阈值时增强，低于阈值时减弱
def bcm_delta(v_i, v_j, theta, eta):
    # TODO 6:
    # 请根据 BCM 规则补全权重变化量:
    #   Δw = eta * v_i * (v_i - theta) * v_j
    # 当 v_i > theta 时增强；当 v_i < theta 时减弱。
    raise NotImplementedError("请补全 bcm_delta")


# 任务一：生成不同 Hebb 变体的更新方向表
def build_hebb_rule_table(eta, gamma2, gamma0, v_theta):
    w0 = 0.5
    v_i_bar = 0.5
    v_j_bar = 0.5

    activity_cases = [
        ("后 ON\n 前 ON", 1.0, 1.0),
        ("后 ON\n 前 OFF", 1.0, 0.0),
        ("后 OFF\n 前 ON", 0.0, 1.0),
        ("后 OFF\n 前 OFF", 0.0, 0.0),
    ]

    rule_names = [
        "标准 Hebb",
        "衰减 Hebb",
        "突触前门控",
        "突触后门控",
        "协变规则",
    ]
```

```
    table = np.zeros((len(rule_names), len(activity_cases)), dtype=int)

    for col, (_, v_i, v_j) in enumerate(activity_cases):
        deltas = [
            standard_hebb_delta(v_i, v_j, eta),
            decay_hebb_delta(w0, v_i, v_j, gamma2, gamma0),
            gated_hebb_delta(v_i, v_j, eta, v_theta, gate_type="pre"),
            gated_hebb_delta(v_i, v_j, eta, v_theta, gate_type="post"),
            covariance_hebb_delta(v_i, v_j, v_i_bar, v_j_bar, eta),
        ]
        table[:, col] = [delta_to_sign(delta) for delta in deltas]

    return table, rule_names, [case[0] for case in activity_cases]



# 任务二：比较无边界、硬边界、软边界下的权重演化
def simulate_boundary_effect(bound_type, steps=60, eta0=0.06, gamma2=0.
06, w0=0.15, w_max=1.0):
    weights = [w0]
    w = w0
    v_i = 1.0
    v_j = 1.0

    for _ in range(steps):
        delta = bounded_hebb_delta(w, v_i, v_j, bound_type=bound_type,
eta0=eta0, gamma2=gamma2, w_max=w_max)
        w = w + delta
        if bound_type in {"hard", "soft"}:
            w = min(w, w_max)
        weights.append(w)

    return np.array(weights)



# 任务三：比较固定阈值和滑动阈值下的 BCM 特异化过程
def simulate_bcm_specialization(use_sliding_threshold, epochs=200, eta=
0.08, theta_fixed=0.10, alpha_theta=1.0, avg_decay=0.03):
    # Group 1 和 Group 2 表示两组不同的突触前输入模式。
    # Group 1 输入强度更高，Group 2 输入强度较低，二者交替出现。
    input_strengths = np.array([1.0, 0.55], dtype=float)
```

```
    weights = np.array([0.35, 0.35], dtype=float)
    avg_output = 0.10


    history = [weights.copy()]
    theta_history = []
    output_history = []


    for epoch in range(epochs):
        active_group = 0 if (epoch // 5) % 2 == 0 else 1
        pre_activity = np.zeros(2, dtype=float)
        pre_activity[active_group] = input_strengths[active_group]
        post_activity = float(np.dot(weights, pre_activity))


        avg_output = (1.0 - avg_decay) * avg_output + avg_decay * post_
activity
        if use_sliding_threshold:
            theta = alpha_theta * avg_output
        else:
            theta = theta_fixed


        for j in range(len(weights)):
            weights[j] += bcm_delta(post_activity, pre_activity[j], the
ta, eta)
        weights = np.clip(weights, 0.0, 1.5)
        history.append(weights.copy())
        theta_history.append(theta)
        output_history.append(post_activity)


    return {
        "weights": np.array(history),
        "theta": np.array(theta_history),
        "output": np.array(output_history),
    }



# 绘制任务一：更新方向表
def draw_rule_table(ax, table, rule_names, activity_labels):
    colors = {
        1: "#2f7d4f",
        0: "#fff7b2",
        -1: "#b91c3d",
    }
    text_map = {-1: "减弱", 0: "不变", 1: "增强"}
```

```
    n_rows, n_cols = table.shape
    ax.set_xlim(-1.7, n_cols)
    ax.set_ylim(0, n_rows + 1.15)
    ax.invert_yaxis()
    ax.axis("off")
    ax.set_title("Hebb 基础变体更新方向（BCM 见下方动态实验）", fontsize=1
7, pad=14)


    ax.text(-0.15, 0.55, "规则", ha="right", va="center", fontsize=12,
weight="bold")
    for j, label in enumerate(activity_labels):
        ax.text(j + 0.5, 0.55, label, ha="center", va="center", fontsiz
e=11, weight="bold")


    for i in range(table.shape[0]):
        y = i + 1
        ax.text(-0.15, y + 0.5, rule_names[i], ha="right", va="center",
 fontsize=12)
        for j in range(table.shape[1]):
            value = int(table[i, j])
            rect = Rectangle((j, y), 1, 1, facecolor=colors[value], edg
ecolor="white", linewidth=1.5)
            ax.add_patch(rect)
            text_color = "white" if value in {-1, 1} else "#222222"
            ax.text(j + 0.5, y + 0.5, text_map[value], ha="center", va=
"center", color=text_color, fontsize=12, weight="bold")



# 绘制任务二：边界约束曲线
def draw_boundary_panel(ax, eta0, gamma2, w_max):
    steps = 60
    time = np.arange(steps + 1)


    none_curve = simulate_boundary_effect("none", steps=steps, eta0=eta
0, gamma2=gamma2, w_max=w_max)
    hard_curve = simulate_boundary_effect("hard", steps=steps, eta0=eta
0, gamma2=gamma2, w_max=w_max)
    soft_curve = simulate_boundary_effect("soft", steps=steps, eta0=eta
0, gamma2=gamma2, w_max=w_max)


    ax.plot(time, none_curve, label="无边界", linewidth=2.0, color="#55
5555")
```

```
    ax.plot(time, hard_curve, label="硬边界", linewidth=2.0, color="#c1
121f")
    ax.plot(time, soft_curve, label="软边界", linewidth=2.0, color="#25
63eb")
    ax.axhline(w_max, color="gray", linestyle="--", linewidth=1.0)
    ax.set_xlabel("时间步")
    ax.set_ylabel("权重值")
    ax.set_title("有界性：无边界、硬边界与软边界")
    ax.grid(alpha=0.25)
    ax.legend()
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)



# 绘制任务三：BCM 固定阈值与滑动阈值对比
def draw_bcm_panel(ax, eta, theta_fixed, alpha_theta, avg_decay):
    fixed = simulate_bcm_specialization(
        use_sliding_threshold=False,
        eta=eta,
        theta_fixed=theta_fixed,
        alpha_theta=alpha_theta,
        avg_decay=avg_decay,
    )
    sliding = simulate_bcm_specialization(
        use_sliding_threshold=True,
        eta=eta,
        theta_fixed=theta_fixed,
        alpha_theta=alpha_theta,
        avg_decay=avg_decay,
    )

    time = np.arange(fixed["weights"].shape[0])
    ax.plot(time, fixed["weights"][:, 0], linestyle="--", color="#2563e
b", linewidth=2.0, label="Group 1 固定阈值（对照）")
    ax.plot(time, fixed["weights"][:, 1], linestyle="--", color="#c1121
f", linewidth=2.0, label="Group 2 固定阈值（对照）")
    ax.plot(time, sliding["weights"][:, 0], linestyle="-", color="#2563
eb", linewidth=2.4, label="Group 1 滑动阈值（BCM）")
    ax.plot(time, sliding["weights"][:, 1], linestyle="-", color="#c112
1f", linewidth=2.4, label="Group 2 滑动阈值（BCM）")
```

```
    ax.set_xlabel("训练轮次")
    ax.set_ylabel("平均权重")
    ax.set_title("BCM 特异化：滑动阈值让神经元偏向强输入组")
    ax.grid(alpha=0.25)
    ax.legend(ncol=2, fontsize=9)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)


def main():
    # 本实验要求修改的参数
    eta = 0.10          # 标准 Hebb、门控 Hebb 和协变规则中的学习率
    eta0 = 0.06         # 硬边界 Hebb 中未达到上界时的固定增强系数
    gamma2 = 0.12       # 衰减 Hebb 和软边界 Hebb 中控制增强速度的系数
    gamma0 = 0.04       # 衰减 Hebb 中控制无刺激衰减速度的系数
    v_theta = 0.50      # 门控 Hebb 中判断活动是否足够强的阈值
    w_max = 3.00        # 有界性实验中允许达到的最大突触权重
    theta_fixed = 0.10 # BCM 对照实验中的固定阈值
    alpha_theta = 1.00 # BCM 滑动阈值系数：theta(t)=alpha_theta * <v_i>
    avg_decay = 0.03    # 输出活动运行平均值的更新速度；越大表示阈值跟随当前
输出变化越快

    table, rule_names, activity_labels = build_hebb_rule_table(
        eta=eta,
        gamma2=gamma2,
        gamma0=gamma0,
        v_theta=v_theta,
    )

    fig = plt.figure(figsize=(13, 10.5))
    fig.patch.set_facecolor("#eef2f7")
    gs = fig.add_gridspec(3, 1, height_ratios=[1.15, 1.0, 1.2])
    plt.subplots_adjust(left=0.12, right=0.95, top=0.91, bottom=0.07, h
space=0.46)

    fig.text(0.08, 0.965, "Hebb 学习规则及其变体", fontsize=20, weight="
bold", color="#333333")
```

```
    ax_table = fig.add_subplot(gs[0])
    ax_boundary = fig.add_subplot(gs[1])
    ax_bcm = fig.add_subplot(gs[2])


    draw_rule_table(ax_table, table, rule_names, activity_labels)
    draw_boundary_panel(ax_boundary, eta0=eta0, gamma2=gamma2, w_max=w_
max)
    draw_bcm_panel(ax_bcm, eta=eta, theta_fixed=theta_fixed, alpha_thet
a=alpha_theta, avg_decay=avg_decay)


    plt.show()



if __name__ == "__main__":
    main()
```

请先补全 standard_hebb_delta、decay_hebb_delta、gated_hebb_delta、covariance_hebb_delta、bounded_hebb_delta 和 bcm_delta，再运行整个脚本。

补全后，重点修改以下参数：

- eta
- eta0
- gamma2
- gamma0
- v_theta
- w_max
- theta_fixed
- alpha_theta
- avg_decay

在报告中分析以下问题：

1. 观察你生成的“Hebb 变体更新方向表”。哪种规则在“后 OFF / 前 OFF”（即 $v_i = 0, v_j = 0$）时会导致权重增强？请对照你补全的代码公式，解释在数学

上是如何产生这个结果的。

2. 观察“突触后门控 Hebb”在图表中的结果。当突触后激活（后 ON）但某一个突触前输入静默（前 OFF）时，权重更新方向是什么？这和竞争与抑制有什么联系？

3. 观察 BCM 曲线中“滑动阈值”模式的图表。当 Group 1（强输入）连续刺激导致 θ 明显上升的这段时间里，Group 2（弱输入）的权重曲线呈现什么趋势？为什么这能促进神经元功能特异化？

二、STDP 学习规则及其变体

STDP 是 Hebb 理论的时间依赖扩展。STDP 不只关心突触前后神经元是否共同活动，还关心突触前、突触后脉冲发放的先后顺序。

时间差定义：

$$\Delta t = t_{post} - t_{pre} \qquad (8)$$

当 $\Delta t > 0$ 时，表示突触前脉冲先于突触后脉冲，对应 LTP；当 $\Delta t < 0$ 时，表示突触后脉冲先于突触前脉冲，对应 LTD。

本实验使用的 pair-based STDP 公式为：

$$\Delta w = \begin{cases} A_+ \exp\left(-\dfrac{\Delta t}{\tau_+}\right), & \Delta t > 0 \\ -A_- \exp\left(\dfrac{\Delta t}{\tau_-}\right), & \Delta t < 0 \\ 0, & \Delta t = 0 \end{cases} \qquad (9)$$

其中，τ_+ 和 τ_- 是时间常数，本实验默认使用 `10.0`。

权重依赖 STDP 中，硬边界通过 Heaviside 阶跃函数限制权重变化：

$$A_+(w_{ij}) = \Theta(w_{max} - w_{ij})\eta_+ \qquad (10)$$

$$A_-(w_{ij}) = \Theta(w_{ij} - w_{min})\eta_- \qquad (11)$$

软边界使权重越接近边界，变化幅度越小：

$$A_+(w_{ij}) = (w_{max} - w_{ij})\eta_+ \qquad (12)$$

$$A_-(w_{ij}) = (w_{ij} - w_{min})\eta_- \qquad (13)$$

本大类实验包含三个任务：

1. 画出 STDP 时间窗口，并观察 Δt 对权重变化的影响；
2. 比较经典 STDP 和软边界权重依赖 STDP 的时间窗口形状；

下面给出 exp7_2.py 的框架代码。

```
import numpy as np
import matplotlib.pyplot as plt



# 配置中文字体，避免绘图时标题或坐标轴出现乱码
plt.rcParams["font.sans-serif"] = [
    "PingFang SC",
    "Hiragino Sans GB",
    "Microsoft YaHei",
    "SimHei",
    "Noto Sans CJK SC",
    "Arial Unicode MS",
    "DejaVu Sans",
]
plt.rcParams["axes.unicode_minus"] = False



# pair-based STDP：根据 pre 和 post 的相对时间差计算一次权重变化
def pair_based_stdp(delta_t, tau_plus=10.0, tau_minus=10.0, a_plus=0.08,
 a_minus=0.08):
    # TODO 7:
    # 请根据 STDP 分段公式补全权重变化量。
    # 约定：
    #   delta_t = t_post - t_pre
    #   delta_t > 0：pre 先于 post，权重增强；
    #   delta_t < 0：post 先于 pre，权重减弱；
    #   delta_t == 0：本实验近似视为无变化。
    raise NotImplementedError("请补全 pair_based_stdp")
```

```python
# 权重依赖 STDP：根据当前权重计算增强或减弱的幅度
def weight_dependent_amplitude(weight, direction, mode, eta_plus, eta_m
inus, w_min=0.0, w_max=1.0):
    # TODO 8：
    # 请补全硬边界和软边界下的权重依赖幅度。
    # 输入：
    #   direction == "ltp" 表示增强，使用 A_+(w)；
    #   direction == "ltd" 表示减弱，使用 A_-(w)；
    #   mode == "classic" 表示不考虑权重依赖，直接返回 eta_plus 或 eta_min
us；
    #   mode == "hard" 表示硬边界；
    #   mode == "soft" 表示软边界：
    #       LTP 时 A_+(w) = (w_max - w) * eta_plus；
    #       LTD 时 A_-(w) = (w - w_min) * eta_minus。
    # 输出：
    #   返回非负幅度值，后续函数会根据 LTP/LTD 决定正负号。
    raise NotImplementedError("请补全 weight_dependent_amplitude")



# 使用权重依赖幅度计算一次 STDP 更新
def weight_dependent_stdp(delta_t, weight, mode, tau_plus=10.0, tau_min
us=10.0, eta_plus=0.05, eta_minus=0.05):
    if delta_t > 0:
        amp = weight_dependent_amplitude(weight, "ltp", mode, eta_plus,
 eta_minus)
        return amp * np.exp(-delta_t / tau_plus)
    if delta_t < 0:
        amp = weight_dependent_amplitude(weight, "ltd", mode, eta_plus,
 eta_minus)
        return -amp * np.exp(delta_t / tau_minus)
    return 0.0



# 重复相同 pre-post 配对，观察权重演化
def simulate_repeated_pair(
    delta_t,
```

```
    mode,
    repeat_count=40,
    initial_weight=0.5,
    tau_plus=10.0,
    tau_minus=10.0,
    eta_plus=0.05,
    eta_minus=0.05,
    w_min=0.0,
    w_max=1.0,
):
    weight = float(initial_weight)
    history = [weight]


    for _ in range(repeat_count):
        delta_w = weight_dependent_stdp(
            delta_t,
            weight,
            mode=mode,
            tau_plus=tau_plus,
            tau_minus=tau_minus,
            eta_plus=eta_plus,
            eta_minus=eta_minus,
        )
        if mode == "classic":
            weight = weight + delta_w
        else:
            weight = np.clip(weight + delta_w, w_min, w_max)
        history.append(weight)


    return np.array(history)



# 绘制任务一: STDP 时间窗口
def draw_stdp_window(ax, tau_plus, tau_minus, a_plus, a_minus):
    delta_range = np.linspace(-40, 40, 401)
    window = np.array(
        [
            pair_based_stdp(
                dt,
                tau_plus=tau_plus,
                tau_minus=tau_minus,
                a_plus=a_plus,
                a_minus=a_minus,
            )
            for dt in delta_range
        ]
```

```
    )

    ax.plot(delta_range, window, color="#333333", linewidth=2.2)
    ax.axhline(0.0, color="gray", linestyle="--", linewidth=1.0)
    ax.axvline(0.0, color="gray", linestyle="--", linewidth=1.0)
    ax.set_xlabel(r"$\Delta t = t_{post} - t_{pre}$")
    ax.set_ylabel(r"$\Delta w$")
    ax.set_title("STDP 时间窗口：pre 先于 post 增强，post 先于 pre 减弱")
    ax.grid(alpha=0.25)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)


# 绘制任务二：权重依赖 STDP 的时间窗口形状
def draw_weight_dependent_window_panel(ax, tau_plus, tau_minus, a_plus,
 a_minus, w_demo, eta_plus, eta_minus, w_min=0.0, w_max=1.0):
    delta_range = np.linspace(-40, 40, 401)
    classic_window = np.array(
        [
            pair_based_stdp(dt, tau_plus=tau_plus, tau_minus=tau_minus,
 a_plus=a_plus, a_minus=a_minus)
            for dt in delta_range
        ]
    )
    soft_window = np.array(
        [
            weight_dependent_stdp(
                dt,
                weight=w_demo,
                mode="soft",
                tau_plus=tau_plus,
                tau_minus=tau_minus,
                eta_plus=eta_plus,
                eta_minus=eta_minus,
            )
            for dt in delta_range
        ]
    )

    ax.plot(delta_range, classic_window, label="经典 STDP", color="#333
333", linewidth=2.1)
    ax.plot(delta_range, soft_window, label=f"软边界 STDP（当前权重 w_ij
={w_demo:.1f}）", color="#2563eb", linewidth=2.1)
    ax.axhline(0.0, color="gray", linestyle="--", linewidth=1.0)
```

```
    ax.axvline(0.0, color="gray", linestyle="--", linewidth=1.0)
    ax.set_xlabel(r"$\Delta t = t_{post} - t_{pre}$")
    ax.set_ylabel(r"$\Delta w$")
    ax.set_title(r"权重依赖 STDP 时间窗口（本实验约定 $\Delta t=t_{post}-
t_{pre}$）")
    ax.grid(alpha=0.25)
    ax.legend()
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)



def main():
    # 本实验要求修改的参数
    tau_plus = 10.0       # LTP 时间常数；越大，pre 先于 post 的增强窗口越
宽
    tau_minus = 10.0      # LTD 时间常数；越大，post 先于 pre 的减弱窗口越
宽
    a_plus = 0.08         # pair-based STDP 中 LTP 的最大权重变化幅度
    a_minus = 0.08        # pair-based STDP 中 LTD 的最大权重变化幅度
    w_demo = 0.80         # 权重依赖 STDP 时间窗口中用于演示的当前权重
    eta_plus = 0.08       # 权重依赖 STDP 中 LTP 的基础更新幅度
    eta_minus = 0.08      # 权重依赖 STDP 中 LTD 的基础更新幅度
    fig = plt.figure(figsize=(13, 8.8))
    fig.patch.set_facecolor("#eef2f7")
    gs = fig.add_gridspec(2, 1, height_ratios=[1.0, 1.05])
    plt.subplots_adjust(left=0.08, right=0.95, top=0.92, bottom=0.08, h
space=0.48)


    fig.text(0.08, 0.97, "STDP 学习规则及其变体", fontsize=20, weight="b
old", color="#333333")


    ax_window = fig.add_subplot(gs[0])
    ax_weight = fig.add_subplot(gs[1])


    draw_stdp_window(ax_window, tau_plus=tau_plus, tau_minus=tau_minus,
 a_plus=a_plus, a_minus=a_minus)
    draw_weight_dependent_window_panel(
        ax_weight,
        tau_plus=tau_plus,
```

```
        tau_minus=tau_minus,
        a_plus=a_plus,
        a_minus=a_minus,
        w_demo=w_demo,
        eta_plus=eta_plus,
        eta_minus=eta_minus,
    )

    plt.show()


if __name__ == "__main__":
    main()
```

请先补全 `pair_based_stdp` 和 `weight_dependent_amplitude`，再运行整个脚本。

补全后，重点修改以下参数：

- `tau_plus`
- `tau_minus`
- `a_plus`
- `a_minus`
- `w_demo`
- `eta_plus`
- `eta_minus`

在报告中分析以下问题：

1. 在经典 STDP 时间窗口实验中，如果你在参数设置中将 tau_plus 改为 20.0，而 tau_minus 保持 10.0，时间窗口的左右两侧（第一象限和第四象限）的衰减曲线会呈现怎样的视觉差异？这意味着什么？

2. 观察权重依赖 STDP 的时间窗口（以 $w_{demo} = 0.80$ 为例）。此时第一象限（LTP 侧）明显被压扁。这是为什么？假设我们修改参数，传入一个极小的当前权重 $w_{demo} = 0.10$，请根据你编写的软边界代码逻辑预测：此时第一象限和第四象限的曲线幅度分别会发生怎样的变化？

7.3 实验要求

1. 完成实验说明中的 2 个大类实验。
2. 补全文中留出的 8 处 `TODO`，保证代码能够正常运行。
3. 至少提交 5 组实验结果图，需覆盖以下内容：
    - Hebb 变体更新方向表；
    - Hebb 有界性权重演化曲线；
    - BCM 固定阈值与滑动阈值权重演化曲线；
    - STDP 时间窗口曲线；
    - 权重依赖 STDP 时间窗口曲线。
4. 提交实验代码文件。
5. 提交一份实验报告或结果说明，报告中必须包含关键图像、参数设置和对应分析文字，并覆盖前文要求完成的现象观察和问题分析，以及回答下面的问题。

实验报告中回答以下问题：

- 在软边界的 Hebb 学习中，当权重逼近最大值 w_{max} 时，权重的增长速度会发生什么变化？请结合生物学中的资源有限性解释其合理性。
- 协变规则和 BCM 规则都引入了平均发放率 $\langle v \rangle$。结合线性代数知识思考，减去均值这个操作，在提取数据的哪些特征（如主成分）时会有帮助？
- 著名的 Hebb 原则说‘共同激发即连接’。STDP 在此基础上增加了怎样的‘时间因果性’要求？如果我们在实验中设置 $\Delta t = 0$，突触权重会如何变化？这在真实的物理世界中可能发生吗？

提交内容如下：

- 实验代码文件，命名为 `exp7_1.py`、`exp7_2.py`；
- 一份实验报告；
- 报告中应包含关键图像、参数设置以及对实验现象的分析。